

Controlador SDN para Equipamentos MikroTik

Licenciatura em Engenharia Informática

Bernardo Guerra de Sá Inácio - 2222402

Pedro Afonso Geraldo Mota - 2230754

Leiria, abril de 2026

Controlador SDN para Equipamentos MikroTik

Licenciatura em Engenharia Informática

Bernardo Guerra de Sá Inácio – 2222402

Pedro Afonso Geraldo Mota - 2230754

Trabalho de Projeto da unidade curricular de Laboratório de Tecnologias de Informação realizado sob a orientação do(a) Professor(a) Doutor(a) Daniel Fuentes.

Leiria, abril de 2026

Dedicatória

Dedico este trabalho à minha família e aos meus amigos, pela compreensão e por estarem sempre presentes, mesmo durante as longas horas dedicadas ao desenvolvimento deste projeto.

Agradecimentos

Gostaria de expressar o meu sincero agradecimento ao meu orientador, o Professor Daniel Fuentes, pela sua constante disponibilidade, orientação rigorosa e partilha de conhecimentos ao longo de todo o desenvolvimento deste projeto.

Resumo

A configuração manual de routers em ambiente empresarial é uma tarefa demorada e propensa a erros. Este projeto apresenta o desenvolvimento de um Controlador SDN (Software-Defined Networking) criado especificamente para equipamentos MikroTik. A solução consiste numa aplicação de ambiente de trabalho, desenvolvida em C# (.NET), que comunica com o router através da sua REST API para automatizar tarefas de rede.

O programa permite gerir de forma visual e intuitiva as interfaces físicas, bridges e redes sem fios (Wi-Fi), incluindo a criação de perfis de segurança. No que toca ao endereçamento, a aplicação facilita a gestão de IPs, servidores DNS e a configuração completa de servidores DHCP através de um sistema de navegação inteligente. Um dos pontos centrais do trabalho é a gestão de VPNs WireGuard, onde a aplicação gera as chaves de segurança no próprio computador e permite exportar a configuração via QR Code. Além disso, foi implementada uma proteção que impede o utilizador de apagar ou alterar rotas automáticas do sistema, garantindo maior estabilidade à rede.

Este projeto demonstra que é possível simplificar a gestão de redes complexas através de software, reduzindo falhas humanas e tornando o trabalho do administrador mais eficiente e seguro.

Palavras-chave: SDN, MikroTik, Automação de Redes, WireGuard, C#, REST API

Abstract

Manually configuring network routers in a business environment is a time-consuming and error-prone task. This project presents the development of an SDN (*Software-Defined Networking*) Controller specifically designed for MikroTik devices. The solution is a desktop application developed in C# (.NET) that communicates with the router through its REST API to automate network tasks.

The program allows for visual and intuitive management of physical interfaces, bridges, and wireless networks (Wi-Fi), including the creation of security profiles. Regarding addressing, the application simplifies IP management, DNS servers, and the complete configuration of DHCP servers through an intelligent navigation system. A core part of the project is the management of *WireGuard* VPNs, where the application generates security keys on the user's computer and allows exporting the configuration via *QR Code*. Furthermore, a safety feature was implemented to prevent users from deleting or modifying automatic system routes, ensuring greater network stability.

This project demonstrates that it is possible to simplify complex network management through software, reducing human error and making the administrator's job more efficient and secure.

Keywords: SDN, MikroTik, Network Automation, WireGuard, C#, REST API

Índice

Dedicatória	1
Agradecimentos	2
Resumo	3
Abstract	4
Lista de Figuras	8
Lista de siglas e acrónimos.....	9
1. Introdução	10
1.1. Contextualização do Problema.....	10
1.2. Objetivos do Projeto.....	10
1.3. Estrutura do Relatório	11
2. Estado da arte e tecnologias utilizadas	12
2.1. O Modelo SDN	12
2.2. Tecnologias de Rede: MikroTik e RouterOS	12
2.3. O Protocolo WireGuard VPN	13
2.4. Tecnologias de Desenvolvimento.....	13
3. Arquitetura da Aplicação	15
3.1. Estrutura do Código (Core vs. UI).....	15
3.2. O Núcleo (MikroTikSDN.Core)	15
3.3. A Interface Gráfica (MikroTikSDN.UI).....	17
3.4. Comunicação REST API e JSON	19
3.5. Gestão de Multissessão em Memória	20
4. Funcionalidades e Desenvolvimento do Controlador.....	21

4.1.	Arquitetura da Interface (UX/UI)	21
4.2.	Gestão de Conetividade (Camada 2)	22
4.3.	Endereçamento e Encaminhamento (Camada 3)	25
4.4.	Gestão de Endereços IP	26
4.5.	Rotas Estáticas e Proteção do Sistema	26
4.6.	Automação de Serviços Essenciais	27
4.7.	Servidor e Cliente DHCP e Gestão de IP Pools	28
4.8.	Configuração de DNS e Proteção de Pedidos Remotos	29
4.9.	Serviços Avançados: WireGuard VPN	30
4.9.1.	Gestão de Servidores e Clientes	30
4.9.2.	Criptografia Local	30
4.9.3.	Automação do Endpoint e Geração de Código QR	31
5.	TESTES E VALIDAÇÃO	32
5.1.	Ambiente de Testes	32
5.2.	Casos de Uso Validados	33
5.2.1.	Gestão e Acesso ao Controlador SDN	33
5.2.2.	Gestão de Múltiplos Dispositivos	35
5.2.3.	Listagem Geral de Interfaces	36
5.2.4.	Gestão de Conetividade Wireless e Segurança	36
5.2.5.	Gestão de Interfaces Bridge e Portas Associadas	38
5.2.6.	Gestão de Endereços IP	40
5.2.7.	Gestão de Encaminhamento (Rotas Estáticas)	41
5.2.8.	Gestão de Serviço DHCP	42
5.2.9.	Gestão de Gamas de Endereços (IP Pools)	44
5.2.10.	Gestão de Serviço DNS	45
5.2.11.	Gestão de VPN (WireGuard) e Acesso Remoto	46
5.3.	Dificuldades Encontradas e Soluções	47

6. Conclusão	50
Bibliografia ou Referências Bibliográficas.....	51

Lista de Figuras

Figura 1-Microtik.Core	16
Figura 2-MicrotikUI.....	18
Figura 3-Ambiente de Testes	32
Figura 4 - Gestão e Acesso ao Controlador SDN.....	34
Figura 5 - Gestão de Múltiplos Dispositivos.....	35
Figura 6 - Listagem de Interfaces.....	36
Figura 7-Wireless.....	37
Figura 8-Perfis de Segurança	37
Figura 9 - Redes Wireless Virtuais	38
Figura 10 – Interfaces Bridges	39
Figura 11 - Associação de Portas á Bridge	40
Figura 12 - Endereços IP.....	41
Figura 13 - Rotas Estáticas.....	42
Figura 14 - DHCP Server.....	42
Figura 15 - DHCP Client	43
Figura 16 - IP atribuído.....	43
Figura 17 - IP Pools	44
Figura 18 - Serviço DNS.....	45
Figura 19 - Wireguard.....	47

Lista de siglas e acrónimos

API: Application Programming Interface

CRUD: Create, Read, Update, Delete

DHCP: Dynamic Host Configuration Protocol

DNS: Domain Name System

ESTG: Escola Superior de Tecnologia e Gestão

GUI: Graphical User Interface

IP: Internet Protocol

LAN: Local Area Network

REST: Representational State Transfer

SDN: Software-Defined Networking

UI/UX: User Interface / User Experience

VPN: Virtual Private Network

WAN: Wide Area Network

1. Introdução

1.1. Contextualização do Problema

No atual panorama tecnológico, as redes de computadores constituem a espinha dorsal de qualquer organização. Contudo, a gestão tradicional de equipamentos de rede, como switches e routers, continua a ser um processo moroso e predominantemente manual. A configuração de dispositivos peça a peça, recorrendo a interfaces de linha de comandos (CLI) ou painéis nativos complexos, exige um elevado grau de especialização técnica e consome tempo valioso aos administradores de sistemas.

Este método manual de gestão apresenta um grande desafio: a forte suscetibilidade a falhas humanas. Um simples erro de sintaxe ou a configuração incorreta de uma rota pode comprometer a conectividade de toda uma empresa, gerando quebras de serviço (downtime) dispendiosas.

Para responder a estas limitações, a indústria tem caminhado no sentido do paradigma de Software-Defined Networking (SDN) e da automação de redes. A separação lógica entre o plano de dados (o equipamento físico) e o plano de controlo (o software de gestão) permite a criação de soluções centralizadas que abstraem a complexidade técnica. Neste contexto, surgiu a necessidade de desenvolver uma ferramenta visual que simplifique a interação com equipamentos MikroTik, permitindo que tarefas complexas e propensas a erro sejam executadas de forma automatizada, rápida e segura.

1.2. Objetivos do Projeto

O objetivo central deste trabalho é o desenvolvimento de um controlador SDN capaz de gerir múltiplos dispositivos MikroTik em simultâneo, permitindo o controlo centralizado de toda a infraestrutura de rede.

Para concretizar este sistema, foram definidos os seguintes objetivos técnicos:

- **Desenvolvimento do Núcleo SDN em C#:** Criar a camada central (*Core*) da aplicação que encapsula toda a lógica de negócio. Este núcleo não se limita à comunicação com a REST API do MikroTik, sendo a verdadeira base estrutural onde residem todos os modelos de dados, serviços e funcionalidades de automação de rede, permitindo ainda a gestão de várias sessões em paralelo.

- **Implementação de Frontend em Windows Forms:** Desenvolver uma interface gráfica visual para o utilizador final que consuma as funcionalidades do núcleo SDN, facilitando a interação e a configuração da rede.
- **Gestão de Conetividade e Serviços:** Implementar ferramentas para a configuração de *Bridges*, Interfaces (físicas e virtuais), Endereçamento IP e serviços essenciais como DNS e DHCP (Servidores, Clientes e *Pools*).
- **Segurança e Acesso Remoto:** Integrar a gestão de VPNs *WireGuard*, incluindo a geração local e automática de chaves criptográficas e a exportação visual de configurações de clientes via *QR Code*.

1.3. Estrutura do Relatório

De forma a apresentar o trabalho desenvolvido de maneira clara e lógica, este relatório encontra-se organizado em seis capítulos fundamentais. Após a presente Introdução, o Capítulo 2 aborda o Estado da Arte e as Tecnologias Utilizadas, detalhando o conceito de SDN e as ferramentas de desenvolvimento escolhidas.

O Capítulo 3 descreve a Arquitetura da Aplicação, focando-se na estruturação do código (separação entre o Core e o Frontend) e na lógica de comunicação API. Segue-se o Capítulo 4, que constitui o núcleo prático do relatório, onde são expostas ao detalhe todas as Funcionalidades e o Desenvolvimento do Controlador. O Capítulo 5 é dedicado aos Testes e Validação, descrevendo o ambiente de ensaio utilizado. Por fim, o Capítulo 6 apresenta as Conclusões do trabalho e sugestões para trabalho futuro.

2. Estado da arte e tecnologias utilizadas

Este capítulo descreve as tecnologias escolhidas para construir o Controlador SDN e as bases técnicas do projeto. A escolha focou-se em ferramentas seguras e eficientes, que permitem automatizar a gestão de equipamentos de rede.

2.1. O Modelo SDN

Numa rede de computadores tradicional, cada router ou switch funciona de forma totalmente independente. Nestes equipamentos clássicos, o "Plano de Dados" (a parte que move a internet de um lado para o outro) e o "Plano de Controlo" (o cérebro que decide para onde a internet vai) estão unidos dentro da mesma caixa física. Esta limitação obriga o administrador de sistemas a aceder e a configurar cada aparelho individualmente, linha a linha.

O modelo SDN (Software-Defined Networking) altera radicalmente esta abordagem, promovendo a separação entre o "cérebro" da rede e o equipamento físico. O controlo passa a ser feito por um software central (o Controlador SDN), enquanto os routers atuam apenas como executantes das ordens que recebem.

Neste projeto, a aplicação desenvolvida atua exatamente como esse controlador central. O utilizador interage com uma interface gráfica simples, que unifica a visão de toda a rede. O software encarrega-se de traduzir as ações do utilizador e de enviar as configurações adequadas para os equipamentos. Isto abstrai a complexidade técnica, reduz a margem para erros humanos e permite gerir múltiplos routers a partir de um único ponto.

2.2. Tecnologias de Rede: MikroTik e RouterOS

O projeto utiliza equipamentos da MikroTik, uma marca amplamente adotada no mercado empresarial por oferecer funcionalidades avançadas de rede a um custo competitivo.

Estes equipamentos funcionam com o sistema operativo RouterOS, que nas suas versões mais recentes disponibiliza uma interface de comunicação chamada REST API. Uma API funciona como uma ponte direta entre sistemas diferentes. Ao contrário das antigas linhas de comandos ou ligações via SSH, a REST API utiliza os métodos padrão da web (como GET, POST, PATCH e DELETE) para ler ou alterar configurações.

Através desta ponte, a aplicação desenvolvida em C# consegue enviar pacotes de dados formatados em JSON (um formato de texto leve e estruturado) diretamente para o router. É este mecanismo fiável que permite à aplicação criar redes Wi-Fi, alterar endereços IP, configurar servidores DHCP ou gerir rotas de forma imediata, sem que o utilizador precise de conhecer os comandos nativos do sistema MikroTik.

2.3. O Protocolo WireGuard VPN

As Redes Privadas Virtuais (VPNs) são fundamentais para garantir o acesso remoto e seguro de funcionários à rede local da empresa. Para implementar este serviço, o projeto integra o protocolo WireGuard.

O WireGuard é um protocolo VPN de nova geração, desenhado para substituir opções mais antigas e pesadas (como o OpenVPN ou IPsec). O seu grande destaque é ser extremamente rápido, leve e consumir poucos recursos de processamento nos equipamentos. A segurança deste protocolo baseia-se numa troca assimétrica de chaves criptográficas: uma Chave Privada (que fica exclusivamente instalada no dispositivo do cliente, como um telemóvel) e uma Chave Pública (que é partilhada com o servidor).

A integração deste protocolo na aplicação permite automatizar totalmente a criação e a gestão destas chaves. O software realiza os cálculos matemáticos complexos em segundo plano, facilitando significativamente o trabalho do administrador, que consegue aprovar novos acessos remotos em segundos sem ter de gerir certificados digitais complicados.

2.4. Tecnologias de Desenvolvimento

A construção do Controlador SDN baseou-se num conjunto de tecnologias da Microsoft e em bibliotecas de código aberto rigorosamente selecionadas:

- **C# e .NET:** A linguagem de programação principal do projeto. O C# permite gerir toda a lógica de negócio da aplicação e processar (desserializar) os dados complexos recebidos em formato JSON. Além disso, a utilização de programação assíncrona garante que a aplicação comunique com a API do *router* sem nunca "congelar" a interface do utilizador.
- **Windows Forms:** A plataforma utilizada para criar a interface gráfica (Frontend). Foi usada de forma dinâmica para desenhar os menus laterais de navegação, tabelas

de dados organizadas e as janelas de edição (*CrudDialogs*) que o utilizador vê no ecrã, garantindo uma experiência de navegação fluída e contextual.

- **BouncyCastle:** Uma biblioteca mundialmente reconhecida, focada em segurança criptográfica. Foi utilizada para gerar o par de chaves do WireGuard (utilizando o algoritmo avançado *Curve25519*) diretamente na memória do computador. Isto garante que a chave privada gerada nunca seja enviada pela rede nem exposta ao *router*, garantindo a segurança máxima do sistema.
- **QRCoder:** Uma biblioteca inteligente que transforma o formato de texto das configurações VPN num código QR bidimensional. Através desta funcionalidade, o ficheiro de configuração é montado visualmente no ecrã. O utilizador só precisa de abrir a aplicação do WireGuard no telemóvel e ler o código com a câmara para ficar instantaneamente ligado à rede da empresa.

3. Arquitetura da Aplicação

O desenvolvimento de um Controlador SDN exige uma arquitetura de *software* bem planeada. Para garantir que a aplicação é rápida, segura e fácil de manter ou expandir no futuro, o código foi estruturado de forma modular, separando claramente o que é "visual" daquilo que é o "motor" do programa.

3.1. Estrutura do Código (Core vs. UI)

Para garantir que a aplicação é rápida, segura, fácil de manter e de escalar no futuro, a arquitetura do projeto foi desenvolvida com base no princípio de separação de responsabilidades (*Separation of Concerns*). O código fonte foi rigorosamente dividido em dois projetos distintos dentro da mesma solução: o **MikroTikSDN.Core** (a biblioteca de classes com a lógica) e o **MikroTikSDN.UI** (a interface visual de interação).

3.2. O Núcleo (MikroTikSDN.Core)

Este é a API do controlador SDN atua como o "motor" da aplicação. Não possui qualquer elemento visual, é totalmente dedicado ao processamento de dados e à comunicação com a rede. Está organizado em três subestruturas principais:

- **Modelos de Dados (*Models*):** Contém as classes C# que representam os recursos da rede (como `BridgeModel`, `DhcpModel`, `WirelessInterface`, entre outros). Estas classes utilizam atributos de desserialização (como o `[JsonPropertyName]`) para mapear automaticamente o texto JSON devolvido pelo router para objetos C# estruturados.
- **Serviços (*Services*):** Foi aplicado o padrão de desenho *Service Pattern*. Em vez de ter um ficheiro gigante a gerir tudo, cada funcionalidade tem o seu próprio serviço dedicado (por exemplo, o `IpPoolService.cs` trata apenas das gamas de IPs, enquanto o `WireGuardService.cs` foca-se exclusivamente na VPN). Todos estes serviços utilizam a classe central `RouterClient` para executar os pedidos HTTP.
- **Tratamento de Exceções (*Exceptions*):** Para garantir a robustez, foi criada uma classe personalizada (`RouterApiException.cs`). Sempre que o router devolve um erro (como credenciais inválidas ou tentativa de apagar um recurso em uso), esta classe

captura o erro, traduz o código HTTP (ex: 400 Bad Request, 403 Forbidden) e envia uma mensagem clara para o utilizador, impedindo que o programa "rebente".

- **RouterManager:** O RouterManager.cs atua como o "maestro" de todo o sistema, sendo o componente responsável por centralizar e coordenar as ligações aos diversos equipamentos MikroTik de forma inteligente. Este gestor possui uma dupla função: primeiro, assegura a persistência de dados, gravando num ficheiro local a lista de dispositivos e as suas credenciais para que o utilizador não tenha de as reintroduzir; segundo, mantém um registo ativo em memória (RAM) de todos os routers autenticados no momento. Esta gestão em memória permite que o administrador alterne instantaneamente entre diferentes dispositivos sem repetir o processo de login. Além de garantir que cada comando seja encaminhado para o equipamento correto, o RouterManager monitoriza a saúde das ligações e assegura uma desconexão segura, limpando dados sensíveis da memória assim que a sessão termina. Esta capacidade transforma a aplicação num verdadeiro controlador SDN, preparado para gerir infraestruturas complexas a partir de um único ponto de controlo.

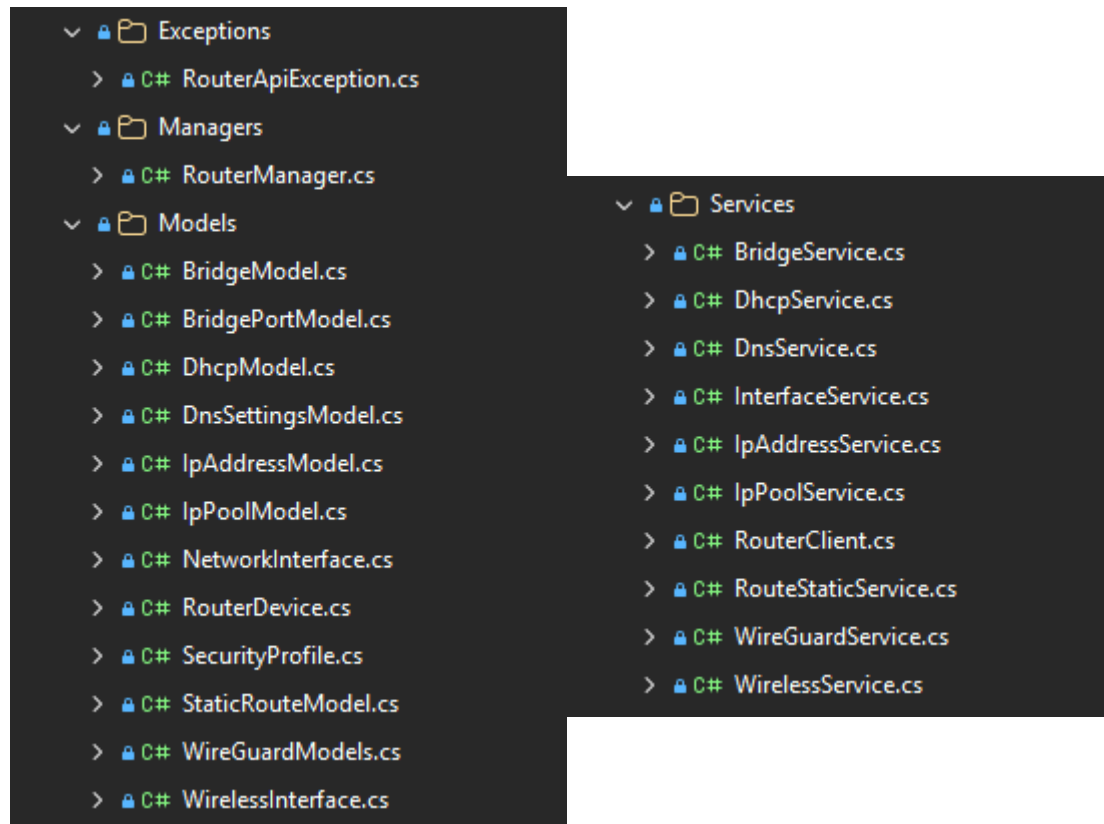


Figura 1-Microtik.Core

3.3. A Interface Gráfica (MikroTikSDN.UI)

Para a construção visual do nosso Controlador SDN, foi utilizada a tecnologia **Windows Forms**, que nos permitiu desenhar todas as janelas e painéis de interação da aplicação. Para garantir que o programa é rápido e fácil de expandir no futuro, a parte visual não foi misturada num único ficheiro, mas sim dividida em três grupos específicos: **Janelas Principais, Painéis Dinâmicos e Janelas Auxiliares.** **Formulários Principais (Forms)** Os formulários são a base da aplicação, atuando como o "esqueleto" do programa.

Janelas Principais da Aplicação

- **Página de Login (LoginForm.cs):** É a porta de entrada e a barreira de segurança da aplicação. Esta página foi desenhada para facilitar a vida ao administrador de redes: além de permitir a ligação manual digitando um IP ou MAC Address, possui uma ferramenta de descoberta de rede (Network Discovery) que varre a rede local e lista os routers MikroTik ativos automaticamente. O utilizador pode também escolher equipamentos cujas credenciais já foram guardadas na memória, poupando o trabalho de as digitar novamente.
- **Página Principal (MainForm.cs):** Depois de o acesso ser validado na página de login, a aplicação avança para a página principal. Este é o verdadeiro controlador central do SDN. É aqui que se encontram as funcionalidades e onde definimos regras, configuramos serviços entre outros. A grande vantagem desta página é que ela não abre janelas novas de cada vez que se clica numa opção. O menu lateral mantém-se fixo, e apenas a zona central do ecrã é alterada consoante a funcionalidade escolhida, o que torna a navegação instantânea.

Painéis Dinâmicos (Pasta *Panel*)

Para preencher a zona central do MainForm, foram criados painéis (peças de interface que se encaixam dentro do formulário principal).

- **DefaultPanel.cs:** É um painel genérico de alta eficiência. Muitas funcionalidades de rede (como Rotas, Endereços IP ou Pools de DHCP) são, na sua essência, apenas listas de dados. Em vez de programar um painel diferente para cada uma destas coisas, o DefaultPanel foi programado para se adaptar. Ele recebe a informação do

núcleo (Core) e desenha uma tabela de dados organizada automaticamente. Isto permitiu poupar linhas de código desnecessárias.

- **InterfacePanel.cs:** Como as portas de rede físicas e as ligações Wi-Fi são mais complexas e precisam de opções visuais específicas (como ativar/desativar portas, ver estatísticas de tráfego ou distinguir entre interfaces principais e virtuais), o painel genérico não era suficiente. Por isso, foi criado este painel dedicado, com botões e layouts feitos à medida apenas para a gestão das interfaces.

Janelas Auxiliares (Pasta *Dialogs*)

As janelas auxiliares são pequenos *pop-ups* que aparecem por cima do programa apenas quando o utilizador precisa de preencher ou editar dados, fechando-se logo a seguir.

- **AddRouterForm.cs:** É a janela invocada a partir do LoginForm. Serve exclusivamente para que o utilizador possa adicionar, nomear (dar um *alias*) e memorizar as credenciais de um novo equipamento MikroTik, gravando-o na lista de acessos rápidos.
- **CrudDialog.cs:** Esta é uma das peças mais avançadas da interface. A sigla CRUD significa *Create, Read, Update, Delete*. O CrudDialog é uma "janela inteligente" de formulário. Se o utilizador clicar em "Adicionar" na secção do DNS, esta janela percebe o contexto e gera apenas uma caixa de texto para o IP do servidor; se clicar em "Adicionar" na VPN, a mesma janela adapta-se e gera campos para chaves públicas e portas. Ao analisar o que é preciso preencher de forma automática, esta

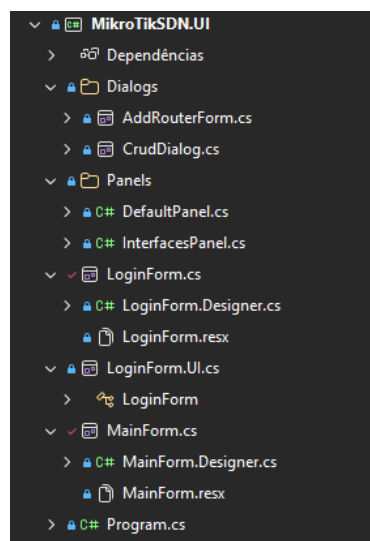


Figura 2-MicrotikUI

janela única substitui a necessidade de programar dezenas de formulários de adição diferentes.

Em resumo, a divisão clara entre o "motor" da aplicação (o núcleo *Core*) e a interface gráfica (desenvolvida em *Windows Forms*) é o que garante a estabilidade e a rapidez deste Controlador SDN. Esta separação técnica, aliada ao uso de painéis dinâmicos e de uma janela inteligente que se adapta automaticamente ao que o utilizador precisa, não só evitou a repetição de código desnecessário, como resultou num sistema leve, muito organizado e perfeitamente preparado para receber novas funcionalidades no futuro.

3.4. Comunicação REST API e JSON

A comunicação entre o controlador SDN e os routers MikroTik é o que permite que a gestão seja feita de forma remota e automática. Em vez de usar comandos de texto manuais, a aplicação utiliza uma linguagem que tanto o computador como o router entendem perfeitamente.

- **O uso da REST API:** A MikroTik disponibiliza uma "porta de comunicação" chamada REST API. A nossa aplicação utiliza o protocolo HTTP (o mesmo usado para navegar na internet) para enviar ordens. Para cada ação, usamos um método específico:
 - **GET:** Para pedir informações ao router (ex: listar endereços IP).
 - **PUT:** Para criar algo novo (ex: configurar um novo servidor DHCP).
 - **PATCH:** Para atualizar uma configuração que já existe.
 - **DELETE:** Para remover uma configuração do equipamento.
- **Formato de Dados JSON:** Toda a informação que viaja entre a aplicação e o router é organizada num formato chamado **JSON**. Este formato funciona como uma "lista de compras" estruturada, onde cada dado tem um nome e um valor (ex: "name": "ether1"). O núcleo da aplicação (Core) tem a inteligência de pegar nos objetos do C# e transformá-los em JSON para enviar ao router, e vice-versa quando recebe uma resposta.

- **Segurança na Transmissão:** Para que o router aceite as ordens, a aplicação injeta automaticamente as credenciais de administrador no cabeçalho (*header*) de cada pedido. Esta comunicação é feita de forma segura, garantindo que apenas o nosso controlador consiga alterar as definições do equipamento.

3.5. Gestão de Multissessão em Memória

Diferente das ferramentas de gestão tradicionais, que normalmente permitem configurar apenas um equipamento de cada vez, este controlador foi desenhado para operar num cenário de múltiplos dispositivos em simultâneo. Esta capacidade é suportada por uma gestão inteligente de sessões diretamente na memória RAM do computador.

- **Dicionário de Sessões Ativas:** O núcleo da aplicação utiliza uma estrutura de dados avançada para armazenar todas as ligações abertas. Quando o utilizador faz *login* num segundo ou terceiro *router*, o sistema não fecha a ligação anterior; em vez disso, guarda os dados de acesso e o canal de comunicação num "cofre" temporário na memória. Isto permite que a troca entre equipamentos seja instantânea, eliminando o tempo de espera de novos carregamentos.
- **Isolamento de Contexto:** Para evitar erros de configuração, o controlador garante que os dados de cada *router* estão totalmente isolados. Ao seleccionar um equipamento na lista da **Página Principal**, o sistema altera o contexto global da aplicação. Isto significa que, embora existam várias sessões abertas, o utilizador apenas visualiza e altera os dados (como Wi-Fi ou DHCP) pertencentes ao *router* que escolheu naquele momento, garantindo que uma ordem nunca é enviada para o destino errado.
- **Persistência e Segurança:** Esta gestão em memória permite que, mesmo que a ligação física com um dos equipamentos sofra uma pequena interrupção, a sessão permaneça "viva" no controlador, tentando recuperar a comunicação automaticamente. No entanto, por questões de segurança, assim que o utilizador encerra a aplicação ou faz *logout*, a memória é imediatamente limpa, garantindo que as credenciais e as chaves de acesso não ficam expostas no sistema após a utilização.

4. Funcionalidades e Desenvolvimento do Controlador

4.1. Arquitetura da Interface (UX/UI)

Navegação Dinâmica e Contextual

A interface do controlador foi desenvolvida com o objetivo de copiar a fluidez de uma *Single Page Application* (SPA), evitando a abertura constante de novas janelas que costuma caracterizar as aplicações tradicionais em *Windows Forms*.

Para isso, a janela principal (MainForm) funciona como uma base que se adapta ao que queremos ver. O menu lateral usa identificadores (Tags, como "iface" ou "dhcp") para saber em que secção o utilizador clicou. A grande vantagem é que, em vez de criarmos um painel novo (UserControl) para cada ecrã diferente, a aplicação reaproveita sempre a mesma tabela visual (DataGridView). Assim, para mudar de menu, basta atualizar os dados que essa tabela mostra (o seu DataSource) usando o método LoadSectionAsync.

A adaptação contextual do ecrã é gerida pelo método UpdateToolbarForSection. Um excelente exemplo desta dinâmica ocorre na secção de DHCP ou WireGuard: a aplicação utiliza variáveis de estado booleanas (como _showDhcpClient ou _showWgPeers) para alterar o comportamento da barra de ferramentas. O mesmo botão físico adapta-se para alternar entre "Ver Clientes" e "Ver Servidores", ocultando botões desnecessários (como o botão de Exportar QR Code, que só fica visível quando estamos no contexto de um Cliente VPN).

Outro ponto de destaque na UX é a criação da janela inteligente CrudDialog. Em vez de desenvolver dezenas de formulários distintos para cada operação de rede, foi programada uma única classe genérica. Esta janela recebe parâmetros em C# (string Label, object Valores). A lógica interna do diálogo analisa o objeto recebido em tempo de execução: se receber uma *string* simples (ex: "192.168.88.1/24"), desenha uma TextBox; se receber um *array* de opções (ex: new[] { "yes", "no" }), desenha automaticamente uma ComboBox. Isto dá ao sistema uma enorme escalabilidade e reduz a redundância de código.

Prevenção de Erros

Para garantir que a aplicação não bloqueia face a respostas inesperadas do *router*, foram implementados vários mecanismos de segurança a nível visual. Primeiro, em ações destrutivas (como remover um IP ou terminar sessão), o utilizador é sempre confrontado com caixas de diálogo de confirmação (MessageBox).

Ao clicar duas vezes num item da tabela para o editar, o programa recupera as informações originais desse registo. Para evitar que a aplicação vá abaixo (*crash*) caso o router envie dados incompletos — como um comentário que não existe ou um gateway vazio —, usamos proteções no código (como o operador ??). Isto garante que, se faltar alguma informação, o programa mostra apenas o campo em branco ou um valor básico, em vez de dar erro

4.2. Gestão de Conetividade (Camada 2)

Interfaces Físicas e Bridges

A gestão de camada 2 (Layer 2) é o principal de qualquer infraestrutura de rede. Neste controlador, essa gestão foi dividida em dois eixos principais: o controlo das interfaces físicas inerentes ao *router* e a criação de *Bridges* (pontes virtuais) que agrupam essas portas num único domínio de *broadcast*.

Na interface gráfica, o utilizador dispõe de um painel dedicado que permite não só visualizar o estado operacional de cada porta física (se está ativa ou desativada), mas também interagir com a agregação dessas portas. Para manter o código organizado, toda a parte de comunicação e envio de ordens para o *router* foi separada e programada nos ficheiros *InterfaceService.cs* e *BridgeService.cs*.

Um dos desafios técnicos superados nesta etapa foi o mapeamento de variáveis. O *framework* .NET utiliza a norma *camelCase* nas suas variáveis (ex: *protocolMode*), no entanto, a API REST do RouterOS exige chaves JSON estritamente formatadas com hífens. Para garantir que o *router* percebe as ordens da nossa aplicação, o código faz uma 'tradução' automática mesmo antes de enviar os dados. Usando um Dictionary, transformamos os nomes das variáveis do nosso C# para o formato exato com hífens que o MikroTik obriga a usar (como "protocol-mode" ou "vlan-filtering"). Para garantir a operação total de leitura, criação, edição e remoção (CRUD) das interfaces e *bridges*, o controlador executa os seguintes pedidos HTTP diretamente para a API REST do MikroTik:

Pedidos de Gestão de Interfaces Físicas (InterfaceService)

- **Listar Interfaces:** GET /rest/interface — Recolhe a lista global de todas as interfaces físicas e virtuais presentes no equipamento.
- **Alterar Estado:** PATCH /rest/interface/{id} — Envia o parâmetro {"disabled": "yes" | "no"} para ativar ou desativar administrativamente uma porta, simulando o comando de *enable/disable*.
- **Apagar Interfaces Virtuais:** DELETE /rest/interface/{id} — Permite a remoção de interfaces dinâmicas/virtuais geradas pelo sistema.

Pedidos de Gestão de Bridges (BridgeService)

- **Listar Bridges:** GET /rest/interface/bridge — Retorna a lista de todas as *bridges* configuradas.
- **Criar Bridge:** PUT /rest/interface/bridge — Cria uma nova *bridge* enviando os parâmetros essenciais: o nome, o protocolo *Spanning Tree* (ex: rstp, stp, mstp, none) e o estado da filtragem de VLANs (vlan-filtering).
- **Editar Bridge:** PATCH /rest/interface/bridge/{id} — Atualiza o nome ou o protocolo de uma *bridge* já existente.
- **Remover Bridge:** DELETE /rest/interface/bridge/{id} — Apaga definitivamente a *bridge* especificada.

Pedidos de Gestão de Portas na Bridge

- **Listar Portas Agrupadas:** GET /rest/interface/bridge/port — Lista todas as interfaces que estão atualmente associadas a uma *bridge*.
- **Adicionar Porta a uma Bridge:** PUT /rest/interface/bridge/port — Associa uma interface física a uma *bridge* específica, permitindo definir o PVID (Port VLAN ID) por defeito (ex: pvid = "1").
- **Editar Atribuição da Porta:** PATCH /rest/interface/bridge/port/{id} — Permite mover a porta para uma *bridge* diferente ou alterar o seu PVID.

- **Remover Porta da Bridge:** DELETE /rest/interface/bridge/port/{id} — Desassocia a interface da *bridge*, devolvendo-lhe a sua independência de roteamento.

Interfaces Wireless VAPs e Perfis de Segurança

A configuração e gestão das interfaces Wireless representam um desafio num controlador SDN, dada a necessidade de coordenar as configurações de *hardware* (rádios físicos) com políticas lógicas de segurança. No nosso controlador, a responsabilidade disto recai sobre a classe `WirelessService.cs`.

O programa não gere apenas as interfaces Wireless normais do equipamento (como a `wlan1`). Ele também permite criar Redes Virtuais (VAPs). Na prática, isto significa que conseguimos usar a mesma antena do *router* para emitir vários nomes de rede (SSIDs) ao mesmo tempo — por exemplo, criar a rede principal da empresa e, com o mesmo equipamento, criar uma rede isolada só para convidados.

Para assegurar a robustez da rede sem fios, implementou-se a gestão isolada dos "Perfis de Segurança" (*Security Profiles*). Em vez de definir palavras-passe diretamente em cada interface, o sistema adota a filosofia do RouterOS: cria-se um perfil de segurança contendo os protocolos (ex: `wpa2-psk`), cifras (ex: `aes-ccm`) e a respetiva chave pré-partilhada, sendo esse perfil posteriormente atribuído à interface pretendida.

Ao nível do processamento, a injeção dos dados é garantida pela junção das variáveis num formato de dicionário (`Dictionary<string, string>`). O código processa internamente caracteres como espaços em branco através de métodos como `authTypes.Replace(" ", "")` antes do envio, prevenindo erros na desserialização pelo *router*.

A comunicação entre a aplicação e a infraestrutura de rede para garantir estas operações decorre mediante os seguintes pedidos REST:

Gestão de Interfaces Sem Fios e VAPs (/rest/interface/wireless)

- **Listar Interfaces Wireless:** GET /rest/interface/wireless — Consulta as interfaces *wireless* presentes, distinguindo as nativas das virtuais pelo campo `master-interface`.
- **Criar Interface Virtual (VAP):** PUT /rest/interface/wireless — Exige a definição da interface rádio subjacente (`master-interface`), o nome da rede (`ssid`) e obriga ao

protocolo mode: "ap-bridge". O perfil de segurança por defeito adotado é o "default", caso o utilizador não o defina.

- **Atualizar Configurações da Interface:** PATCH /rest/interface/wireless/{id} — Permite a redefinição de vários parâmetros vitais do rádio de forma agregada, como a frequência (frequency), modo de operação (mode), país de emissão (country) ou o perfil de segurança (security-profile).
- **Alterar Estado Administrativo:** PATCH /rest/interface/wireless/{id} — Desativa ou ativa a emissão da rede enviando um simples estado ("disabled": "yes" ou "no").
- **Remover Interface Virtual:** DELETE /rest/interface/wireless/{id} — Elimina de forma definitiva um VAP do equipamento.

Gestão dos Perfis de Segurança (/rest/interface/wireless/security-profiles)

- **Consultar Perfis Atuais:** GET /rest/interface/wireless/security-profiles — Lista todas as políticas de segurança disponíveis no *router*.
- **Criar Novo Perfil de Segurança:** PUT /rest/interface/wireless/security-profiles — Cria um novo perfil indicando as chaves dinâmicas (mode: "dynamic-keys"), protocolos de autenticação e injeta a palavra-passe (wpa2-pre-shared-key e wpa-pre-shared-key).
- **Editar Perfil Existente:** PATCH /rest/interface/wireless/security-profiles/{id} — Atualiza os parâmetros do perfil. A aplicação do método REST PATCH assegura que apenas as *ciphers* ou palavras-passe alteradas sejam processadas, sem comprometer os restantes dados do perfil.
- **Apagar Perfil de Segurança:** DELETE /rest/interface/wireless/security-profiles/{id} — Remove o perfil do equipamento (a operação é validada pelas exceções do núcleo caso o perfil esteja em uso num VAP).

4.3. Endereçamento e Encaminhamento (Camada 3)

Após o estabelecimento da conectividade na Camada 2, a atribuição de endereços lógicos e a definição dos caminhos de rede (rotas) constituem o núcleo funcional do equipamento. Este processamento lógico (Camada 3) foi implementado na aplicação através de dois serviços dedicados: o IpAddressService.cs e o RouteService.cs.

4.4. Gestão de Endereços IP

A gestão do endereçamento num *router* exige não só a definição do endereço lógico em notação CIDR (ex: 192.168.88.1/24), mas também a sua associação obrigatória a uma interface subjacente (física ou virtual).

O `IpAddressService` encarrega-se desta operação, empacotando o endereço e a interface destino num dicionário que é posteriormente processado pelo cliente HTTP. A comunicação entre o controlador SDN e o MikroTik baseia-se nas seguintes chamadas de API:

- **Listar Endereços IP:** GET `/rest/ip/address` — Obtém a lista completa de endereços IP configurados no *router*, incluindo os gerados dinamicamente (como os atribuídos por PPPoE ou DHCP Clients).
- **Adicionar Endereço:** PUT `/rest/ip/address` — Regista um novo endereço na rede. Requer dois parâmetros obrigatórios: o `address` (com máscara CIDR) e a interface correspondente.
- **Editar Endereço Existente:** PATCH `/rest/ip/address/{id}` — Permite alterar o endereço IP ou migrá-lo para uma interface diferente, sem necessidade de o remover previamente.
- **Alterar Estado Administrativo:** PATCH `/rest/ip/address/{id}` — Inativa (disabled: "yes") ou ativa o endereço, isolando temporariamente a rede sem perder a configuração.
- **Remover Endereço:** DELETE `/rest/ip/address/{id}` — Apaga definitivamente o endereço IP manual do equipamento.

4.5. Rotas Estáticas e Proteção do Sistema

O encaminhamento de tráfego (Routing) é vital para garantir que os pacotes de dados alcançam as redes externas, sendo gerido pelo `RouteService`.

Uma funcionalidade crítica desenvolvida neste serviço foi a adoção rigorosa das nomenclaturas do RouterOS. Como o MikroTik requer hifenização (ex: "dst-address"), o serviço faz o mapeamento das variáveis do C# (`dstAddress`) para o formato nativo da API REST, garantindo que não há falhas na criação da rota.

Na parte da Proteção do Sistema, tivemos de lidar com uma regra do MikroTik: o *router* tem rotas criadas por nós (estáticas) e rotas criadas automaticamente por ele (dinâmicas), sendo que o equipamento bloqueia qualquer tentativa de apagar estas rotas dinâmicas.

Se o utilizador tentar apagar uma rota automática por engano, o *router* vai devolver um erro. Em vez de deixar a nossa aplicação ir abaixo (*crash*) quando isso acontece, criámos uma proteção no código chamada RouterApiException. Na prática, o que isto faz é 'apanhar' o erro técnico do *router* e transformá-lo num aviso simples no ecrã. Assim, o utilizador percebe que não pode apagar aquela rota, e o programa continua a funcionar perfeitamente.

As interações REST relativas às rotas efetuam-se da seguinte forma:

- **Listar Rotas:** GET /rest/ip/route — Retorna a tabela global de roteamento (FIB - *Forwarding Information Base*).
- **Criar Rota Estática:** PUT /rest/ip/route — Cria uma nova rota, exigindo o endereço de destino (dst-address), o ponto de saída (gateway) e, opcionalmente, a distância administrativa (distance) para a priorização de caminhos.
- **Atualizar Rota:** PATCH /rest/ip/route/{id} — Modifica os parâmetros da rota (Destino, *Gateway*, Distância) e injeta descrições organizacionais através do parâmetro comment.
- **Alterar Estado:** PATCH /rest/ip/route/{id} — Inativa ou reativa uma rota específica (disabled: "yes" | "no").
- **Remover Rota:** DELETE /rest/ip/route/{id} — Emite um pedido de eliminação. Se for uma rota estática (manual), é removida; se for uma rota dinâmica, a ação é barrada pela API e devidamente tratada pelo controlador.

4.6. Automação de Serviços Essenciais

A orquestração de serviços de rede essenciais, nomeadamente o DHCP (*Dynamic Host Configuration Protocol*) e o DNS (*Domain Name System*), constitui um dos pontos fulcrais da aplicação desenvolvida. A configuração manual destes serviços no sistema RouterOS exige múltiplos passos encadeados, complexidade essa que o controlador mitiga através dos serviços DhcpService, IpPoolService e DnsService.

4.7. Servidor e Cliente DHCP e Gestão de IP Pools

A implementação do serviço DHCP foi conjunta: o controlador permite gerir o equipamento tanto como Cliente (para receber endereços do ISP) como Servidor (para fornecer IPs à rede local).

O grande destaque desta secção é a forma como automatizámos o Servidor DHCP. Configurar isto à mão no MikroTik dá algum trabalho porque o *router* obriga-nos a criar três coisas separadas: a lista de IPs que vamos distribuir (*Pool*), as regras da rede com os *gateways* (*Network*), e finalmente ligar tudo isto à interface física.

Para facilitar a vida ao utilizador, a nossa aplicação pede tudo numa única janela de forma muito simples. No entanto, por trás dos panos, o nosso código encarrega-se de fazer o trabalho pesado, dividindo o processo e enviando os seguintes pedidos pela ordem certa ao equipamento:

- **Gestão de Pools (IpPoolService):**
 - GET /rest/ip/pool — Consulta as gamas de IPs existentes.
 - PUT /rest/ip/pool — Cria uma nova gama enviando o nome (name) e o intervalo (ranges, ex: "192.168.88.10-192.168.88.254").
 - PATCH /rest/ip/pool/{id} e DELETE /rest/ip/pool/{id} — Edita ou remove a Pool respetiva.
- **Gestão do Servidor e Rede DHCP (DhcpService):**
 - PUT /rest/ip/dhcp-server/network — O controlador cria a lógica de rede subjacente ao DHCP, enviando o bloco de endereços (address), a porta de saída padrão (gateway) e os servidores DNS sugeridos aos clientes (dns-server).
 - GET /rest/ip/dhcp-server — Lista os servidores a operar no equipamento.
 - PUT /rest/ip/dhcp-server — Instancia o servidor final associando-o a uma interface, atribuindo-lhe um nome e vinculando a address-pool previamente criada. O parâmetro "disabled" = "no" é forçado para garantir o arranque imediato.

- PATCH /rest/ip/dhcp-server/{id} e DELETE /rest/ip/dhcp-server/{id} — Permitem atualizar a associação da interface/pool ou apagar o serviço.
- **Gestão do Cliente DHCP (DhcpClient):**
 - PUT /rest/ip/dhcp-client — Associa uma interface (geralmente a WAN) para solicitar um endereço dinâmico, enviando configurações cruciais como use-peer-dns (para adotar o DNS do fornecedor) e add-default-route (para criar a rota de internet).

4.8. Configuração de DNS e Proteção de Pedidos Remotos

Ao contrário das interfaces ou rotas, que são listas de múltiplos objetos, o DNS no RouterOS atua como uma configuração global de sistema. Esta característica arquitetural do MikroTik gerou um desafio interessante no consumo da API.

Em recursos globais, o uso do método PATCH falha com um erro 400 Bad Request (missing resource identifier), uma vez que não existe um "ID" de linha para editar. Para contornar esta limitação de forma eficiente, o DnsService foi concebido para invocar o comando nativo de /set através do método POST. Adicionalmente, esta abordagem tem a vantagem de não utilizar filtros de dados nulos: ao enviar uma *string* vazia ("") na carga útil (Payload) do JSON, o MikroTik interpreta a instrução como um comando explícito para apagar aquele valor específico.

A interação via API REST para o DNS resume-se a:

- GET /rest/ip/dns — Obtém os servidores DNS primários (servers), configurações de DNS sobre HTTPS (use-doh-server) e o estado da *cache* (cache-size).
- POST /rest/ip/dns/set — Executa a alteração global. Um parâmetro crítico integrado neste envio é o allow-remote-requests, que dita se o *router* atua como um servidor de resolução de nomes para os clientes da rede. Este botão de acesso rápido é vital para proteger o *router* contra ataques de amplificação de DNS quando exposto à rede pública.

4.9. Serviços Avançados: WireGuard VPN

Respondendo ao requisito de implementação de um serviço de VPN, o controlador SDN integra o protocolo WireGuard. Distinguindo-se por operar ao nível do *kernel* e utilizar criptografia de última geração, o WireGuard é mais rápido e leve do que protocolos clássicos como o OpenVPN ou IPsec. Este serviço encontra-se na classe WireGuardService.cs e constitui a funcionalidade tecnicamente mais avançada da aplicação.

4.9.1. Gestão de Servidores e Clientes

A configuração do WireGuard baseia-se em dois pilares: a interface emissora (Servidor) e a lista de dispositivos com permissão de acesso (*Peers* / Clientes). O controlador processa as configurações enviando os seguintes pedidos REST:

- Gestão da Interface (Servidor):
 - GET /rest/interface/wireguard — Consulta as interfaces VPN ativas.
 - PUT /rest/interface/wireguard — Cria um novo servidor, definindo o nome e a porta de escuta UDP (listen-port).
 - PATCH e DELETE /rest/interface/wireguard/{id} — Permite a edição da porta ou remoção completa da interface (com respetivos alertas de segurança para evitar cortes acidentais).
- Gestão de Peers (Clientes):
 - GET /rest/interface/wireguard/peers — Lista todos os clientes registados.
 - PUT /rest/interface/wireguard/peers — Regista um novo dispositivo no servidor. O controlador envia a chave pública gerada (public-key), a interface de destino, o IP estático atribuído dentro da VPN (allowed-address) e o nome do dispositivo (comment).

4.9.2. Criptografia Local

Um dos pontos mais fortes da nossa segurança é a forma como criamos as chaves da VPN (Pública e Privada). Em vez de mandarmos o *router* fazer esse trabalho, a nossa aplicação gera as chaves localmente, fazendo os cálculos diretamente na memória (RAM) do

computador. Para garantir que isto é feito de forma totalmente segura, utilizámos uma biblioteca especializada e muito reconhecida chamada BouncyCastle.

O método `GenerateKeyPair()` executa três passos críticos:

1. Gera 32 *bytes* aleatórios de alta entropia através do `RandomNumberGenerator`.
2. Aplica as restrições matemáticas de *clamping* obrigatórias do protocolo WireGuard sobre esses *bytes*.
3. Calcula a Chave Pública com base na Chave Privada utilizando a função `X25519.ScalarMultBase`.

Esta arquitetura assegura que **a chave privada nunca é transmitida pela rede nem guardada no router**. Ela permanece exclusivamente no "cofre" temporário da aplicação (`_vaultPrivado`) apenas o tempo suficiente para ser transformada num QR Code, garantindo um sistema de Zero Conhecimento (*Zero-Knowledge*) por parte do equipamento de rede.

4.9.3. Automação do Endpoint e Geração de Código QR

Para que um telemóvel se consiga ligar à rede da empresa a partir de casa, precisa de saber o 'caminho' na internet, ou seja, o endereço público (Endpoint) do router. A nossa aplicação descobre isto sozinha usando o método `GetPublicEndpointAsync`. Primeiro, ela pergunta ao próprio router (`GET /rest/ip/cloud`) se ele tem um endereço oficial configurado (o DDNS da MikroTik). Se não tiver, a aplicação tem um 'Plano B' inteligente: vai à internet e usa o site `api.ipify.org` para descobrir qual é o IP público real da rede.

Depois de ter todas as peças (o endereço, as chaves e o IP do cliente), o programa escreve a configuração da VPN automaticamente. A grande vantagem aqui é a facilidade de uso: usamos uma biblioteca chamada `QRCoder` para transformar esse texto num Código QR no ecrã. Na prática, isto pega numa configuração super complexa e torna-a simples: o administrador regista o cliente no PC, aparece o código, e a pessoa só tem de apontar a câmara do telemóvel para ficar logo ligada à VPN em segurança.

5. TESTES E VALIDAÇÃO

Para garantir que a aplicação cumpre todos os requisitos exigidos no enunciado e está pronta para ser utilizada num ambiente real de administração de redes, foi elaborada uma bateria de testes rigorosa. O objetivo deste capítulo é demonstrar, passo a passo, como cada funcionalidade é operada na interface e validar se os comandos enviados pelo nosso controlador são corretamente interpretados e aplicados pelo router MikroTik.

5.1. Ambiente de Testes

Para a realização destes testes, foi montado um ambiente laboratorial controlado:

- **Equipamento Físico e Software:** Utilizou-se um router MikroTik físico a correr o sistema operativo RouterOS (versão 7), ligado através de um cabo de rede ou por wifi a um computador com sistema operativo Windows, onde a nossa aplicação foi executada.
- **Metodologia de Validação (WinBox):** A validação de cada teste foi feita com recurso ao **WinBox** (o software oficial de gestão da MikroTik). O método consistiu em executar a ação na nossa aplicação e, imediatamente a seguir, observar no WinBox se a alteração ocorria em tempo real no equipamento, provando o sucesso da comunicação via API.

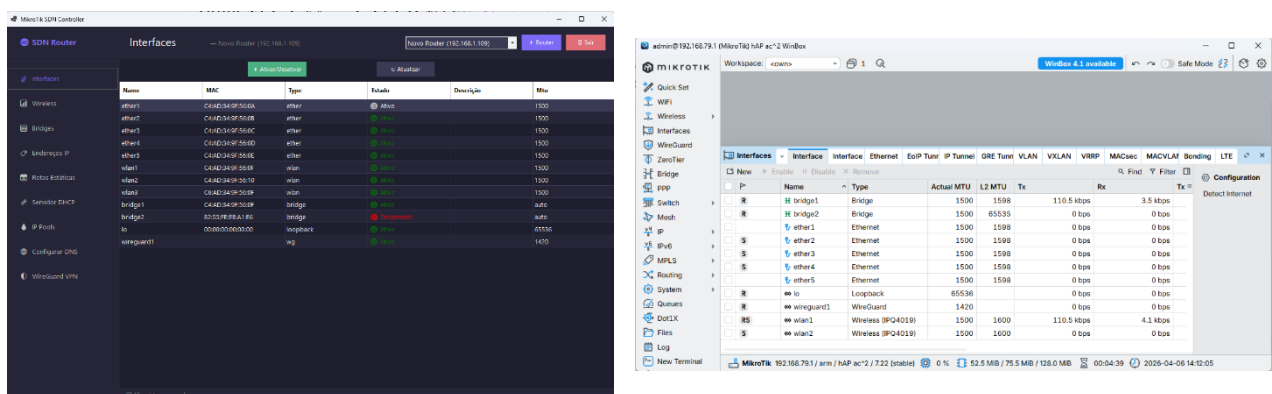


Figura 3-Ambiente de Testes

5.2. Casos de Uso Validados

Abaixo, detalhamos o procedimento exato de utilização para cada uma das funcionalidades exigidas, explicando a interação do utilizador com a interface.

5.2.1. Gestão e Acesso ao Controlador SDN

O primeiro grande teste realizado foi ao sistema de entrada na aplicação, que oferece duas formas distintas de adicionar equipamentos, garantindo versatilidade ao administrador.

Acesso manual e Autenticação Direta

- **Como utilizar (Passo a Passo):** No ecrã inicial, o utilizador que já possui os dados do equipamento pode preencher manualmente os campos: "Endereço IP/MAC", "Utilizador" e "Palavra-passe". Após preencher, clica no botão "Ligar".
- **O que a aplicação faz:** O núcleo valida se os campos estão preenchidos e tenta estabelecer uma ligação direta via HTTPS/API. Se as credenciais estiverem corretas, a sessão é iniciada.
- **Validação:** Testámos o acesso manual com IPs locais e remotos, confirmando que o controlador estabelece a ligação.

Descoberta Automática de Dispositivos

- **Como utilizar (Passo a Passo):** Caso o utilizador não saiba o endereço do router, clica no botão "Procurar Dispositivos". A interface abre uma lista onde, após alguns segundos, aparecem todos os equipamentos MikroTik detetados na rede. O utilizador apenas precisa de clicar no dispositivo desejado para que os dados sejam preenchidos automaticamente.
- **O que a aplicação faz:** A aplicação envia um pacote de "varrimento" (broadcast) pela rede local. O RouterManager interpreta as respostas dos dispositivos MikroTik que devolvem a sua identidade (IP e Endereço MAC), apresentando-os de forma organizada ao utilizador.
- **Validação:** O teste foi bem-sucedido apresentado assim os dispositivos na rede e o seu acesso.

Opção de Memorização de Dispositivos

- **Como utilizar (Passo a Passo):** Em baixo na janela de login, existe uma caixa de verificação (checkbox) com a opção "Memorizar Dispositivo". Antes de clicar em ligar, o utilizador ativa esta opção.
- **O que a aplicação faz:** Se a ligação for bem-sucedida, o RouterManager não só abre a sessão, como grava um registo no ficheiro de configuração local (router_db.json). Na próxima vez que o programa abrir, este router aparecerá numa de dispositivos guardados.
- **Lógica Técnica:** Isto evita que o administrador tenha de repetir a busca automática ou a inserção manual em cada utilização, tornando o fluxo de trabalho muito mais profissional e rápido.
- **Validação:** A funcionalidade é confirmada pela presença dos dispositivos na lista lateral esquerda da interface e pela gravação simultânea dos dados no ficheiro router_db.json.

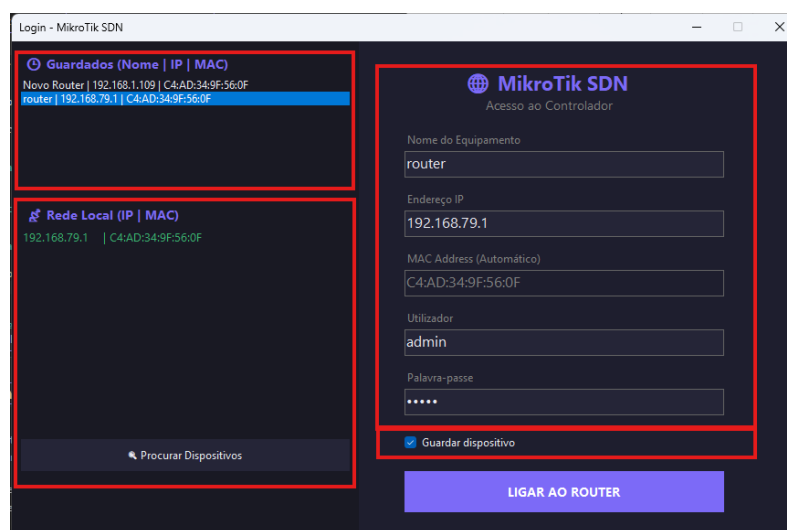


Figura 4 - Gestão e Acesso ao Controlador SDN

5.2.2. Gestão de Múltiplos Dispositivos

- **Como utilizar (Passo a Passo):** Na interface principal, no canto superior direito, o utilizador encontra as ferramentas de gestão de sessão. Ao lado da caixa de seleção (onde se escolhe qual o router a configurar), existe o botão "+ Router". Ao clicar neste botão, a aplicação abre novamente a janela de login, permitindo introduzir os dados de um novo equipamento. Uma vez autenticado, o novo router é adicionado à caixa de seleção. Para terminar a gestão de um dispositivo específico, o utilizador seleciona-o e clica no botão "Sair", localizado na mesma área.
- **O que a aplicação faz:** Sempre que um novo dispositivo é inserido através do botão "+ Router", o RouterManager cria uma nova instância de ligação sem derrubar as anteriores. A caixa de seleção no topo serve como o seletor de "foco": ao mudar o router nessa caixa, a aplicação faz o varrimento de dados apenas para esse equipamento. O botão de "Sair" encarrega-se de fechar a ligação ativa e remover o nome do dispositivo da lista de seleção.
- **Lógica Técnica:** A centralização destes controlos no canto superior direito melhora a usabilidade, permitindo que o administrador salte entre diferentes redes (ex: "Sede" e "Filial") com apenas dois cliques, mantendo o controlo total sobre quais sessões devem permanecer abertas ou ser encerradas.
- **Validação:** A funcionalidade é validada pela abertura bem-sucedida da janela de login ao clicar em "+ Router", pela atualização da lista de seleção após a entrada de um novo equipamento e pelo encerramento individual da sessão ao clicar em "Sair".

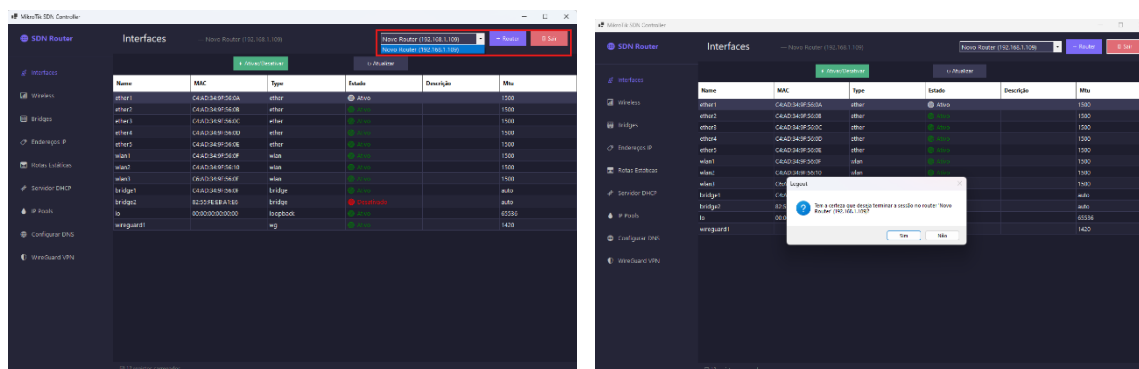
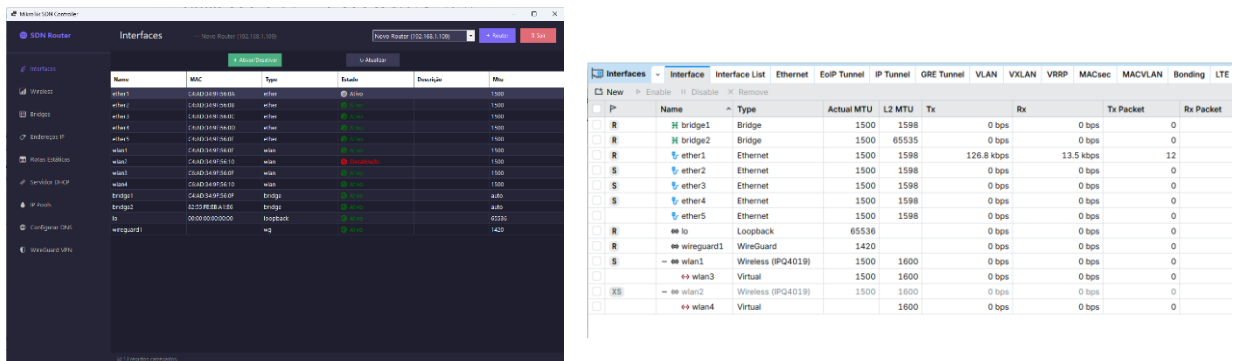


Figura 5 - Gestão de Múltiplos Dispositivos

5.2.3. Listagem Geral de Interfaces

- **Como utilizar (Passo a Passo):** No menu lateral principal, o utilizador clica na página "**Interfaces**". Esta página é dedicada exclusivamente às ligações físicas e lógicas gerais do equipamento.
- **O que a aplicação faz:** Através de um pedido ao endpoint /interface, a aplicação preenche uma tabela com todas as portas Ethernet, SFP e virtuais. São exibidas colunas com o Nome, o Tipo e o Estado (Up/Down) de cada ligação.
- **Validação:** A listagem foi validada comparando-a com o menu "Interface List" do WinBox, confirmando que todas as portas físicas estão presentes e com o estado atualizado.



Name	MAC	Type	Status	Description	MTU
ether1	14:00:14:00:00:00	ether	Up		1500
ether2	14:00:14:00:00:00	ether	Up		1500
ether3	14:00:14:00:00:00	ether	Up		1500
ether4	14:00:14:00:00:00	ether	Up		1500
ether5	14:00:14:00:00:00	ether	Up		1500
wlan1	14:00:14:00:00:00	wlan	Up		1500
wlan2	14:00:14:00:00:00	wlan	Up		1500
wlan3	14:00:14:00:00:00	wlan	Up		1500
bridge1	14:00:14:00:00:00	bridge	Up		auto
bridge2	14:00:14:00:00:00	bridge	Up		auto
lo	00:00:00:00:00:00	loopback	Up		4292
wireguard1		wg	Up		1421

Name	Type	Actual MTU	L2 MTU	Tx	Rx	Tx Packet	Rx Packet
bridge1	Bridge	1500	1598	0 bps	0 bps	0	0
bridge2	Bridge	1500	65535	0 bps	0 bps	0	0
ether1	Ethernet	1500	1598	126.8 kbps	13.5 kbps	12	0
ether2	Ethernet	1500	1598	0 bps	0 bps	0	0
ether3	Ethernet	1500	1598	0 bps	0 bps	0	0
ether4	Ethernet	1500	1598	0 bps	0 bps	0	0
ether5	Ethernet	1500	1598	0 bps	0 bps	0	0
lo	Loopback	65536	1598	0 bps	0 bps	0	0
wireguard1	WireGuard	1420	0 bps	0 bps	0 bps	0	0
wlan1	Wireless (802.11n)	1500	1600	0 bps	0 bps	0	0
wlan2	Wireless (802.11n)	1500	1600	0 bps	0 bps	0	0
wlan3	Wireless (802.11n)	1500	1600	0 bps	0 bps	0	0
wlan4	Wireless (802.11n)	1500	1600	0 bps	0 bps	0	0

Figura 6 - Listagem de Interfaces

•

5.2.4. Gestão de Conetividade Wireless e Segurança

Listagem e Configuração de Interfaces Wireless

- **Como utilizar (Passo a Passo):** O utilizador acede à página "**Wireless**" no menu lateral. Na página é possível visualizar as interfaces Wireless existentes. Para configurar uma interface wireless pressiona duas vezes na interface pretendida na tabela e preenche os campos necessários para o tipo de configuração pretendida.
- **O que a aplicação faz:** A aplicação comunica com o endpoint /interface/wireless. Ao editar, o sistema envia os novos parâmetros para o router; ao clicar nos botões de estado, o controlador envia um comando imediato para ligar ou desligar o sinal de wireless.

- **Validação:** O sucesso é comprovado pela alteração do estado na tabela e pela disponibilidade (ou desaparecimento) do sinal Wi-Fi detetado por dispositivos externos.

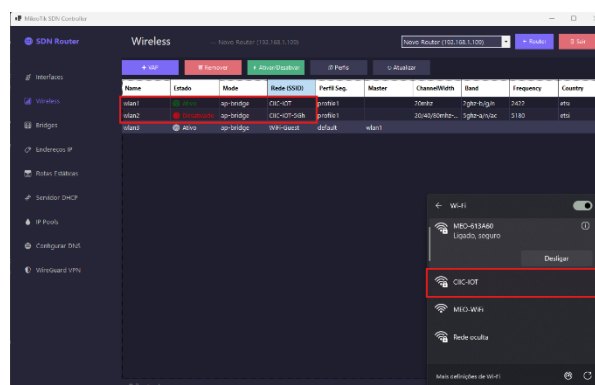


Figura 7-Wireless

Perfis de Segurança (Security Profiles)

- **Como utilizar (Passo a Passo):** Na página de "Wireless" encontra-se uma página responsável para os perfis de Segurança, o utilizador pode gerir as chaves de acesso. Para criar uma nova proteção, clica em "+ Perfil", define um nome para o perfil, escolhe o tipo de autenticação (ex: WPA2 PSK) e insere a palavra-passe. É também possível editar perfis existentes ou apagar os que já não são necessários.
- **O que a aplicação faz:** O sistema gere o CRUD completo no endpoint /interface/wireless/security-profiles. Este perfil fica então disponível como uma opção de escolha na configuração das interfaces wireless.
- **Validação:** A prova de funcionamento é a integração deste perfil na página de Wireless e a exigência da password correta ao tentar ligar um dispositivo à rede.

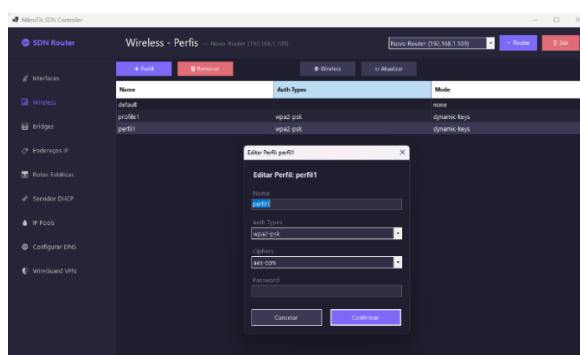
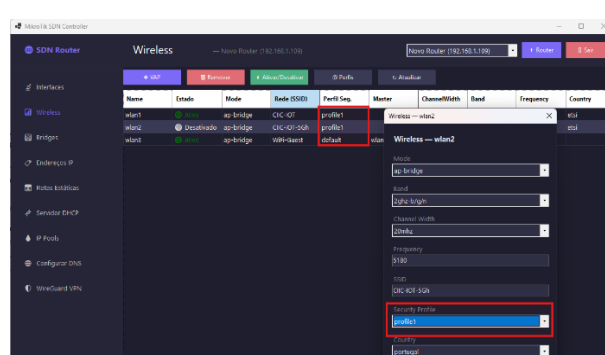


Figura 8-Perfis de Segurança



Redes Wireless Virtuais (Virtual AP)

- **Como utilizar (Passo a Passo):** Dentro da página Wireless, o utilizador tem a opção de criar uma "**Interface Virtual**". Deve seleccionar uma interface física "mestre" (ex: wlan1), atribuir um novo SSID (ex: "WiFi-Convidados") e escolher um perfil de segurança específico para esta rede virtual.
- **O que a aplicação faz:** A aplicação envia um comando para criar uma nova interface do tipo Virtual AP. Isto permite que um único rádio físico emita múltiplos nomes de rede com seguranças e permissões diferentes.
- **Validação:** Verificou-se a criação da interface virtual no WinBox (como sub-item da interface física) e a propagação simultânea de dois nomes de rede diferentes a partir do mesmo router.

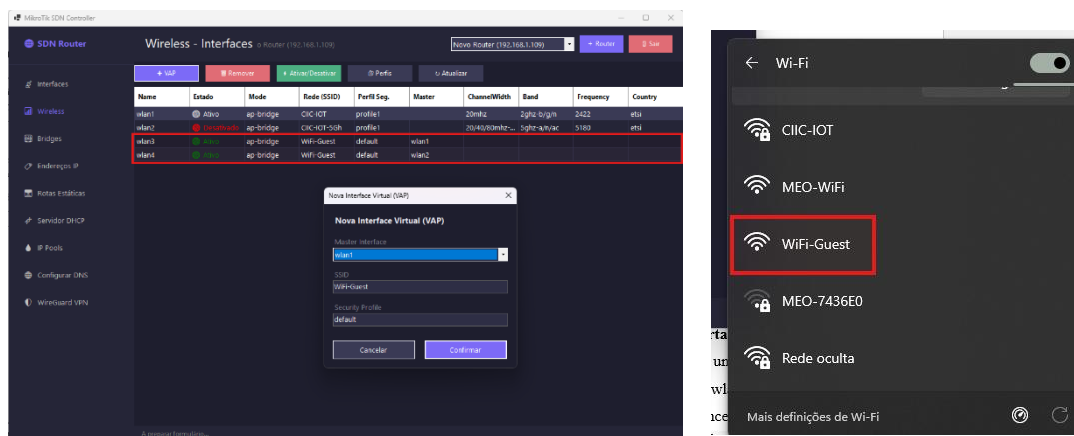


Figura 9 - Redes Wireless Virtuais

5.2.5. Gestão de Interfaces Bridge e Portas Associadas

Listagem e Gestão de Interfaces Bridge

- **Como utilizar (Passo a Passo):** O utilizador acede à página "**Bridge**" no menu lateral, onde é apresentada a lista das bridges existentes. Para criar uma nova bridge, clica em "+ Bridge", introduz um nome (ex: Bridge-Escritorio), o protocolo se for necessário e guarda. O utilizador pode também editar o nome de uma Bridge existente ou apagar uma que já não seja necessária, utilizando os botões de ação na tabela.
- **O que a aplicação faz:** A aplicação comunica com o endpoint /interface/bridge. Ao criar ou apagar, executa os pedidos HTTP necessários para instruir o sistema RouterOS a gerar ou remover essa interface lógica de rede, garantindo que as regras de dependência (não apagar se tiver portas) são respeitadas.

- **Validação:** A criação e eliminação foram validadas abrindo o menu "Bridge" no WinBox, confirmando que a interface lógica aparece e desaparece em tempo real consoante os comandos enviados pela nossa aplicação.

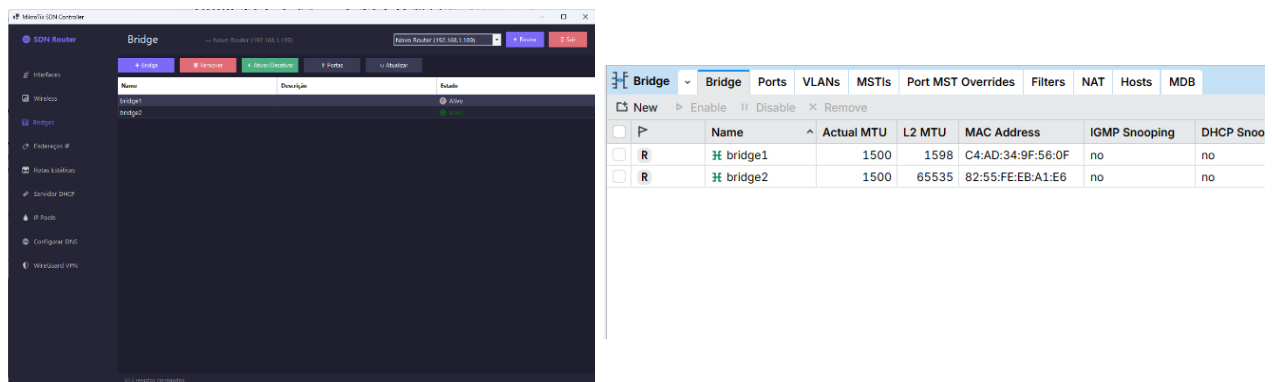


Figura 10 – Interfaces Bridges

Associação de Portas às Bridges (Bridge Ports)

- **Como utilizar (Passo a Passo):** Na secção dedicada a "Portas", o utilizador visualiza quais as interfaces físicas já agrupadas. Para adicionar uma nova ligação, clica em "+ Porta", selecciona a interface física (ex: ether2 ou wlan1) num menu pendente e, de seguida, selecciona a Bridge à qual esta deve pertencer. Se precisar de remover uma porta da Bridge para lhe devolver a independência, basta seleccioná-la na lista e clicar em "Apagar".
- **O que a aplicação faz:** O controlador interage com o endpoint /interface/bridge/port. Em vez de configurar IPs diretamente na porta, a aplicação diz ao Mikrotik para mapear (associar) essa interface física à Bridge escolhida, unificando o tráfego de camada 2.
- **Validação:** Confirmou-se no WinBox, no separador "Ports" da secção Bridge, que a interface física seleccionada foi adicionada com sucesso e ganhou a identificação de estado "S" **Slave** nas interfaces provando que está corretamente associada à Bridge.

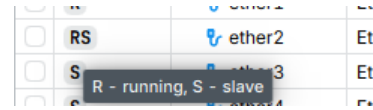
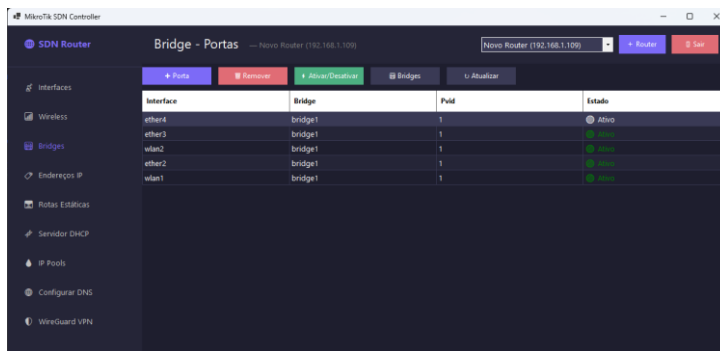


Figura 11 - Associação de Portas à Bridge

5.2.6. Gestão de Endereços IP

Listagem, Atribuição e Gestão de IPs (IP Addresses)

- **Como utilizar (Passo a Passo):** O utilizador acede à página "IP Addresses" no menu lateral, onde visualiza a listagem de todos os endereços configurados no equipamento. Para atribuir um novo IP, clica em "+ IP", introduz o endereço em conjunto com a máscara de rede na notação CIDR (ex: 192.168.10.1/24) e selecciona, através de um menu pendente, a interface (física ou Bridge) à qual esse IP vai pertencer. O administrador pode também editar um IP existente para corrigir o endereço/interface ou utilizar o botão "Apagar" para remover IPs obsoletos.
- **O que a aplicação faz:** A aplicação comunica diretamente com o endpoint /ip/address. Antes de enviar o comando, o sistema valida se a estrutura do IP inserida pelo utilizador está no formato correto. Em seguida, executa o pedido (POST para criar, PATCH/PUT para editar, DELETE para apagar) injetando o endereço IP na camada de rede (Layer 3) da interface especificada.
- **Validação:** A validação foi feita em duas partes: visualmente, confirmando que o endereço e a sub-rede (Network) apareceram corretamente calculados no menu "IP > Addresses" do WinBox; e de forma prática, ligando um computador à interface configurada e executando um comando *ping* para o novo IP do *router*, confirmando a conectividade com sucesso.

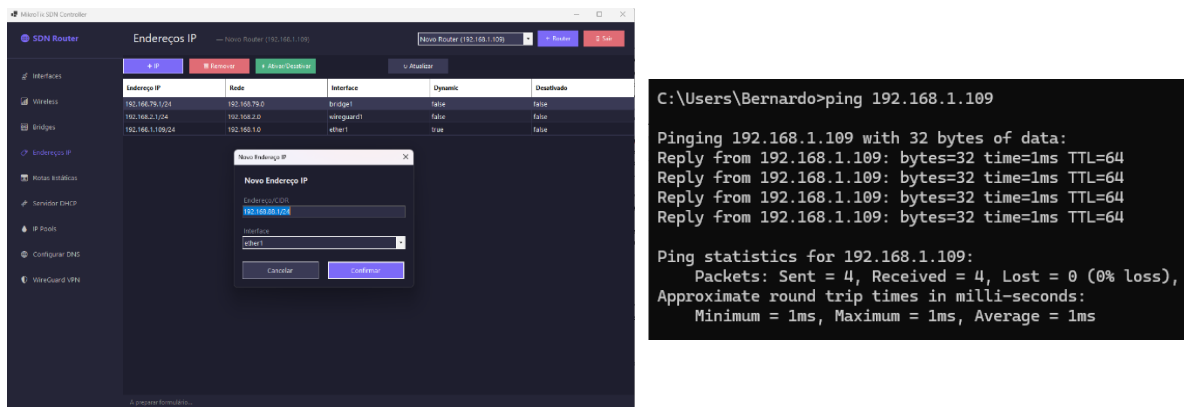


Figura 12 - Endereços IP

5.2.7. Gestão de Encaminhamento (Rotas Estáticas)

Listagem, Criação e Gestão de Rotas Estáticas

- Como utilizar (Passo a Passo):** O utilizador acede à página "**Rotas Estáticas**" no menu lateral para visualizar a tabela de encaminhamento do equipamento. Para definir um novo caminho para o tráfego (como a saída para a internet), clica em "+ Rotas", introduz o endereço de destino no campo respetivo (ex: 0.0.0.0/0 para a rota padrão/internet) e preenche o endereço IP do Gateway (o equipamento seguinte que vai receber o tráfego, ex: 192.168.1.254). Através dos botões na tabela, o administrador pode também editar o Gateway de uma rota já existente ou apagar rotas que tenham deixado de ser necessárias.
- O que a aplicação faz:** O núcleo da aplicação comunica com o endpoint /ip/route. Ao enviar estes dados, o controlador injeta regras na tabela de encaminhamento lógico (Layer 3) do RouterOS. Isto instrui o MikroTik sobre qual a porta ou IP exato por onde deve enviar os pacotes de dados destinados a redes que ele não conhece diretamente.
- Validação:** A operação foi validada visualmente através do menu "IP > Routes" no WinBox, confirmando que a nova rota estática foi adicionada.

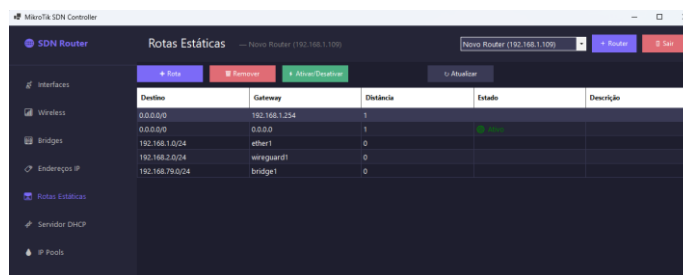


Figura 13 - Rotas Estáticas

5.2.8. Gestão de Serviço DHCP

Listagem e Gestão de DHCP Server, DHCP Client e Acesso às Pools

- **Como utilizar (Passo a Passo):** O utilizador acede à página "**Servidor DHCP**" no menu lateral, que centraliza as funcionalidades de gestão dinâmica de IPs. Nesta interface, o utilizador tem acesso a três áreas principais:
 1. **DHCP Server:** Para criar um novo servidor, o utilizador clica no botão "+ Server". Este botão atua como um assistente inteligente (semelhante ao DHCP Setup nativo do MikroTik). O utilizador apenas precisa de preencher um formulário único com os dados essenciais: interface, o endereço de rede, a gama de endereços e os servidores DNS.

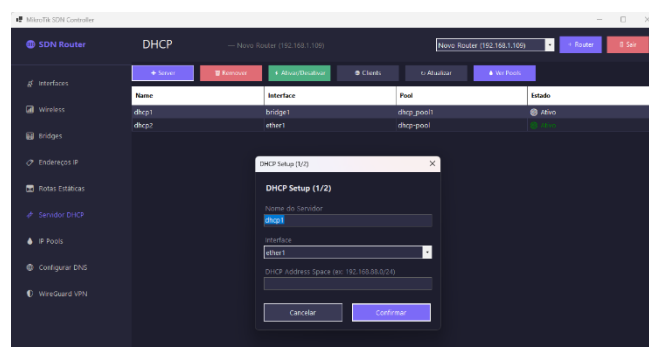


Figura 14 - DHCP Server

2. **DHCP Client:** Para configurar o router de forma a receber um IP automaticamente do fornecedor de internet (ISP). O utilizador seleciona a interface de entrada (ex: ether1) e ativa a opção.

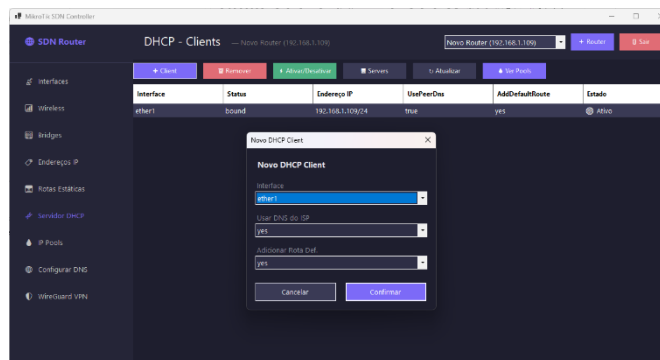


Figura 15 - DHCP Client

3. **IP Pools:** Existe um botão de atalho dedicado nesta página que encaminha diretamente o utilizador para a gestão de Pools (onde se definem as gamas de IPs a distribuir, cujo funcionamento será detalhado na sua própria secção).
- **O que a aplicação faz:** O controlador comunica de forma independente com os endpoints `/ip/dhcp-server` e `/ip/dhcp-client`. No caso do Server, a aplicação automatiza e injeta as regras para que o Mikrotik escute os pedidos de novos dispositivos na rede local e lhes atribua IPs com base na Pool associada. No caso do Client, instrui a porta WAN do router a emitir um pedido de IP para a rede externa.
 - **Validação:** A validação visual foi efetuada no WinBox, acedendo a "IP > DHCP Server" e "IP > DHCP Client", confirmando que ambos os serviços foram criados corretamente. A validação confirma que são atribuídos IPs de acordo DHCP Server e o range estabelecido e são atribuído IPs a clientes DHCP.

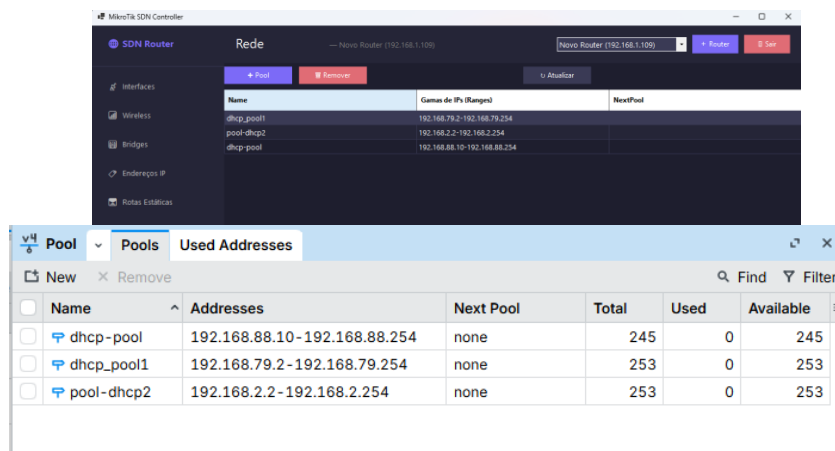
Endereços IP — Novo Router (192.168.1.109)				
+ IP Remover Ativar/Desativar Atualizar				
Endereço IP	Rede	Interface	Dynamic	Estado
192.168.79.1/24	192.168.79.0	bridge1	false	Ativo
192.168.2.1/24	192.168.2.0	wireguard1	false	Ativo
192.168.1.109/24	192.168.1.0	ether1	true	Ativo

Figura 16 - IP atribuído

5.2.9. Gestão de Gamas de Endereços (IP Pools)

Listagem e Configuração de Pools

- **Como utilizar (Passo a Passo):** O utilizador acede à página "IP Pools" (seja pelo menu lateral ou através do botão de atalho na página de DHCP). Aqui, é apresentada uma tabela com todas as gamas de IPs configuradas. Para criar uma nova gama, o utilizador clica em "Adicionar", define um nome (ex: pool-vpn ou pool-wifi) e introduz o intervalo de IPs (ex: 192.168.10.50-192.168.10.100). Através da interface, é também possível **editar** o intervalo de endereços de uma pool existente ou **apagar** gamas que já não estejam associadas a nenhum serviço ativo.
- **O que a aplicação faz:** O controlador interage com o endpoint /ip/pool. Esta funcionalidade é essencial, pois as Pools funcionam como a "base de dados" de endereços para outros serviços. Quando o utilizador cria ou edita uma Pool, a aplicação garante que o formato do intervalo (separado por hífen) é respeitado antes de enviar o comando para o RouterOS. Embora o assistente "+ Server" do DHCP crie pools automaticamente, esta página permite um controlo granular e manual sobre esses intervalos.
- **Validação:** A validação visual foi realizada no WinBox, acedendo ao menu "IP > Pool", onde se confirmou que as gamas criadas pela aplicação apareciam com o nome e intervalo de IPs exatos. A validação técnica foi confirmada ao associar uma destas Pools a um servidor DHCP ou serviço de VPN e verificar que os dispositivos clientes recebiam apenas endereços contidos dentro do intervalo definido na Pool.



Name	Gamas de IPs (Ranges)	NextPool
dhcp_pool1	192.168.79.2-192.168.79.254	
pool-dhcp2	192.168.2.2-192.168.2.254	
dhcp-pool	192.168.88.10-192.168.88.254	

Name	Addresses	Next Pool	Total	Used	Available
dhcp-pool	192.168.88.10-192.168.88.254	none	245	0	245
dhcp_pool1	192.168.79.2-192.168.79.254	none	253	0	253
pool-dhcp2	192.168.2.2-192.168.2.254	none	253	0	253

Figura 17 - IP Pools

Configuração de Servidores DNS e Acesso Remoto

- [illegible]



5.2.11. Gestão de VPN (WireGuard) e Acesso Remoto

Configuração de Túneis, Endereçamento e Exportação (QR Code e .conf)

- **Como utilizar (Passo a Passo):** O utilizador acede à página "**WireGuard VPN**" para configurar o acesso remoto seguro. O processo segue três etapas cruciais:
 1. **Configuração do Servidor:** O utilizador cria a interface WireGuard e, de seguida, deve navegar até à página de **Endereços IP** para atribuir um endereço IP específico a essa interface (ex: 10.0.0.1/24), garantindo que o túnel tenha uma identidade na camada 3.
 2. **Abertura de Porta (Port Forward):** Para permitir que o tráfego da VPN vindo da internet chegue ao router, o administrador deve garantir a configuração de uma regra de Port Forwarding (NAT) na porta UDP seleccionada.
 3. **Gestão e Exportação para Clientes (Peers):** Na secção de Peers, o utilizador adiciona os dispositivos clientes. Para distribuir as configurações de acesso sem a necessidade de trocas manuais de chaves públicas, a aplicação oferece duas soluções automáticas: o administrador selecciona o cliente na lista e clica em "**Gerar QR Code**" (ideal para smartphones) ou no botão "**Exportar .conf**" (ideal para computadores portáteis e desktops).
- **O que a aplicação faz:** O controlador gere toda a complexidade da criptografia de chaves assimétricas. Ao solicitar a configuração (via QR Code ou ficheiro), a aplicação compila os dados necessários: o IP do cliente, o endpoint do servidor e as chaves de encriptação, fazendo com que o cliente "adote" automaticamente a configuração. Na opção de exportação, o sistema gera e guarda no computador do utilizador um ficheiro de texto estruturado com a extensão .conf, que é o formato oficial e universalmente aceite pelas aplicações cliente do WireGuard (Windows, macOS, Linux).

- **Validação:** A validação foi efetuada testando os dois métodos de distribuição. Para dispositivos móveis, a leitura do QR Code na app do telemóvel importou o perfil de imediato. Para computadores, é usado o ficheiro exportado .conf que mostra as propriedades necessárias para conexão ao vpn.

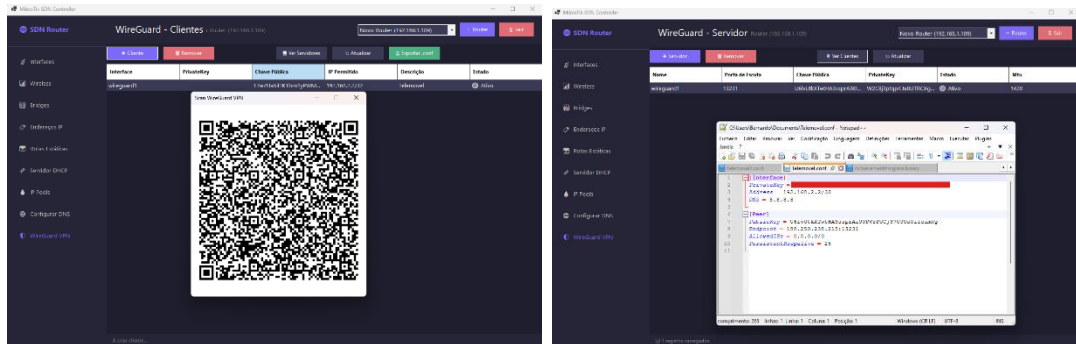


Figura 19 - Wireguard

5.3. Dificuldades Encontradas e Soluções

Durante o desenvolvimento e validação do controlador SDN, surgiram vários desafios técnicos, resultantes da complexidade de integrar uma interface gráfica (C#) com a API do sistema operativo RouterOS v7. Destacam-se as três principais dificuldades e as respetivas soluções implementadas:

Falhas nas Tabelas devido a Propriedades Nuláveis (Null Reference Exceptions)

- **O Desafio:** Durante os testes de listagem (em páginas como Interfaces, IP Addresses ou Routes), a aplicação encerrava inesperadamente ("estoirava") quando o utilizador clicava em certas linhas da tabela.
- **A Causa:** Verificou-se que a API do MikroTik devolve estruturas JSON dinâmicas. Certos parâmetros existem para algumas interfaces, mas não para outras (por exemplo, um campo opcional ou dinâmico). Como as classes de modelo (Modelos de Dados) na nossa aplicação estavam à espera de valores estritos e obrigatórios, a tentativa de ler um valor inexistente gerava um erro de objeto nulo.
- **A Solução:** Foi necessário refatorar o código fonte, alterando as classes de mapeamento para utilizarem tipos anuláveis (ex: string?, int?). Adicionalmente, implementaram-se validações preventivas (null-checks) no momento de preencher as linhas da tabela, garantindo que se a API não enviasse um dado, a interface gráfica apresentaria apenas um espaço em branco, mantendo a estabilidade da aplicação.

Integração do Serviço DNS

- **O Desafio:** A configuração do servidor DNS revelou-se inconsistente com a lógica do resto da aplicação, dificultando a implementação dos botões de guardar e aplicar.
- **A Causa:** A maioria dos recursos no MikroTik (como IPs, Rotas ou DHCP) funciona como uma lista gerida por comandos CRUD (Create, Read, Update, Delete) no formato REST. No entanto, o menu /ip/dns no RouterOS é um painel de configuração global e estático. A tentativa de lhe adicionar parâmetros como se fosse uma lista resultava em erros de sintaxe por parte da API.
- **A Solução:** Foi necessário criar um método exclusivo no controlador (RouterManager) para o DNS, utilizando o comando de edição global (set) em vez de tentar adicionar novos registos (add). A lógica teve também de ser ajustada para processar corretamente as listas de IPs separadas por vírgulas e converter a opção Allow Remote Requests para o formato booleano aceite pela API do MikroTik.

Automação do WireGuard e Rejeição de Chaves no QR Code

- **O Desafio:** O objetivo de simplificar a ligação VPN utilizando um QR Code esbarrou num problema de compatibilidade: ao ler o código, o smartphone (através da app oficial do WireGuard) recusava a configuração ou falhava o estabelecimento do túnel.
- **A Causa:** O protocolo WireGuard é extremamente rigoroso com a criptografia. Inicialmente, havia dificuldade em estruturar o texto para o QR Code e, sobretudo, em obrigar o dispositivo cliente a adotar uma Chave Privada (Private Key) gerada e atribuída pela nossa aplicação, em vez de ser o cliente a gerar a sua própria chave. Qualquer erro no formato Base64 ou na estrutura do ficheiro virtual resultava em rejeição.
- **A Solução:** A arquitetura do gerador de configurações WireGuard foi revista. Implementámos a geração automática de um par de chaves válido na nossa aplicação. Em seguida, desenhamos um algoritmo que formata o texto do QR Code de forma rigorosa e exata, injetando a chave privada gerada para o cliente, a chave pública do servidor, o Endpoint (IP Público:Porta) e os AllowedIPs. Com esta precisão no

código, o smartphone passou a adotar as chaves corretamente, aceitando o QR Code à primeira tentativa.

6. Conclusão

O objetivo principal deste projeto foi criar uma aplicação para gerir redes de forma centralizada para equipamentos MikroTik. A ideia era simplificar o trabalho de configuração através de uma interface visual fácil de usar e automática. Depois das fases de planeamento, desenvolvimento e de todos os testes feitos, é possível concluir que todos os objetivos propostos na introdução foram alcançados com sucesso.

A aplicação desenvolvida cumpriu todos os requisitos técnicos exigidos. Provou ser capaz de procurar e gerir vários routers ao mesmo tempo, guardando os dados de acesso para que o utilizador não tenha de os escrever repetidamente. O programa consegue listar, criar, editar e apagar as configurações essenciais, comunicando de forma direta e sem falhas com o sistema do router.

Durante os testes, confirmámos que todas as áreas funcionam corretamente: a gestão de Interfaces, Wireless, Bridges, Endereços IP, Rotas, DHCP e DNS.

Além disso, a segurança e o acesso remoto também foram garantidos com a configuração da VPN WireGuard. Tornámos este serviço muito mais fácil de usar ao criar ficheiros automáticos e um código QR para os telemóveis. Isto evita que o utilizador tenha de configurar chaves de segurança à mão, tornando a aplicação muito mais amigável.

A nível de programação, os problemas que surgiram durante o desenvolvimento foram resolvidos. Foram criadas soluções para lidar com erros de comunicação ou dados em falta, garantindo que o programa é seguro e não se fecha sozinho se ocorrer um problema.

Em resumo, o projeto resultou numa ferramenta prática e de confiança, unindo o que foi aprendido sobre Programação e Redes. A aplicação faz exatamente o que prometeu: gere a rede num só lugar de forma simples, cumprindo o seu objetivo inicial com sucesso.

Bibliografia ou Referências Bibliográficas

MikroTik. (n.d.). *RouterOS v7 REST API*. MikroTik Wiki/Help. Acedido em 2026, de <https://help.mikrotik.com/docs/display/ROS/REST+API>

MikroTik. (n.d.). *WireGuard in RouterOS*. MikroTik Wiki/Help. Acedido em 2026, de <https://help.mikrotik.com/docs/display/ROS/WireGuard>

Setup MikroTik WireGuard VPN for Android (Remote Access)

<https://www.youtube.com/watch?v=okihqaMni-w&t=1s>

Configurar VPN Wireguard no celular Android

<https://www.youtube.com/watch?v=El6MW9Eyaok>